

P3176R1

The Oxford variadic comma

Published Proposal, 2024-11-22

This version:

<https://eisenwave.github.io/cpp-proposals/oxford-variadic-comma.html>

Author:

[Jan Schultke<janschultke@gmail.com>](mailto:janschultke@gmail.com)

Audience:

CWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

[eisenwave/cpp-proposals](https://github.com/eisenwave/cpp-proposals)

Abstract

Deprecate ellipsis parameters without a preceding comma. The syntax `(int...)` is incompatible with C, detrimental to C++, and easily replaceable with `(int, ...)`.

Table of Contents

1 Revision history

1.1 Changes since R0

2 Introduction

2.1 History and C compatibility

3 Motivation

3.1 T.....

4 Impact on the standard

5 Impact on existing code

6 Design considerations

6.1 Why deprecate, not remove?

6.2 What about `(...)`?

6.3 *parameter-declaration-clause*

7 Proposed wording

8 Acknowledgements

References

Normative References

Informative References

NOTE: In this proposal, the term *ellipsis parameter* refers to non-template variadic parameters, i.e. "C-style varargs", i.e. as in `(/* */, ...)`.

§ 1. Revision history

§ 1.1. Changes since R0

- rebase on [\[N4981\]](#)
- inline "Editorial changes" and "Normative changes" subsections in [§ 7 Proposed wording](#)
- reorder proposed expansions of *parameter-declaration-clause*
- update an existing paragraph instead of adding a new one

§ 2. Introduction

[\[P1219R2\]](#) "Homogeneous variadic function parameters" proposed a new feature, where the declarator `(int...)` would be interpreted as a homogeneous function template parameter pack. As part of this, it made it mandatory to separate an ellipsis parameter from preceding parameters using a comma.

The proposal did not pass, but the change to commas achieved strong consensus by EWGI in Kona 2019.

EWGI Poll: Make vararg declaration comma mandatory.

SF	F	N	A	SA
10	6	1	0	0

Going back to 2016, [\[P0281R0\]](#) "Remove comma elision in variadic function declarations" also proposed to make the variadic comma mandatory. Due to its impact on existing code, the proposal did not pass.

My proposal continues where [\[P1219R2\]](#) and [\[P0281R0\]](#) have left off. However, I call for deprecation, not for removal, resulting in the following behavior:

```
// OK, function template parameter pack
template<class... Ts> void a(Ts...);
// OK, abbreviated function template parameter pack
void b(auto...);
// OK, ellipsis parameter, compatible with C
void c(int, ...);
void d(...);
8 // Deprecated, ellipsis parameter, ill-formed in C
9 void e(int...);
10 void f(int x...);
    // Ill-formed, but unambiguous
    void g(int... args);
```

[\[P1161R3\]](#) "Deprecate uses of the comma operator in subscripting expressions" has followed the same strategy. It deprecated `array[0, 1]` in C++20 which freed up the syntax for [\[P2128R6\]](#) "Multidimensional subscript operator" in C++23.

§ 2.1. History and C compatibility

The active version of the C standard ISO/IEC9899:2018 permits functions which accept a variable number of arguments. Such functions must have a *parameter-type-list* containing at least one parameter, and the ellipsis parameter must be comma-separated:

```
parameter-type-list:  
    parameter-list  
    parameter-list , ...
```

This syntax is unchanged since C89. C has never permitted the comma to be omitted.

Such ellipsis parameters were originally introduced in C++ along with function prototypes, but without a separating comma. Only `int printf(char*...)` would have been well-formed in pre-standard C++, unlike `int printf(char*, ...)`.

For the purpose of C compatibility, C++ later allowed the separating comma, resulting in the syntax (unchanged since C++98):

```
parameter-declaration-clause:  
    parameter-declaration-listopt ...opt  
    parameter-declaration-list , ...
```

With the introduction of function template parameter packs in C++11, there arose a syntactical ambiguity for a declarator `(T...)` between:

- a parameter pack of type `T...` and
- a single `T` followed by an ellipsis parameter.

Currently, this ambiguity is resolved in favor of function template parameter packs, if that is well-formed.

§ 3. Motivation

The declarator syntax `(int...)` interferes with future standardization. This has already impacted [\[P1219R2\]](#) and will impact any future proposal which attempts to utilize this syntax.

Furthermore, variadic function templates are a much common and flexible way of writing variadic functions in C++. Many users associate `(int...)` with a pack, not with an ellipsis parameter. Not parsing this syntax as a parameter pack is confusing and has potential for mistakes:

```
template<class Ts>  
void f(Ts...); // well-formed, but equivalent to (Ts, ...)
```

Users who are not yet familiar with variadic templates can easily make the mistake of omitting `...` after `class`. Another possible mistake can occur when writing generic lambdas or abbreviated function templates:

```
// abbreviated variadic function template
void g(auto ...args);
// abbreviated non-variadic function template with ellipsis parameter
void g(auto args...);
```

These two forms look awfully similar and are both well-formed with different meaning. The latter should be deprecated.

Lastly, as explained above, only the declarator `(int, ...)` is compatible with C, not `(int...)`. The latter syntax is arguably pointless nowadays and only exists for the sake of compatibility with pre-standard C++.

§ 3.1. `T.....`

C++ also allows `(auto.....)` or `(T.....)`, which is a declarator containing a function template parameter pack, followed by an ellipsis parameter. Such *parameter-declaration-clauses* are confusing because conceptually, they consist of two separate constructs, but the syntax strongly suggests that all `.` apply to `auto` or `T`.

NOTE: In other words, the declarator `(auto.....)` is equivalent to `(auto..., ...)`.

NOTE: Some users believe that this construct is entirely useless because one cannot provide any arguments to the ellipsis parameter. However, arguments can be provided if `T...` is template type pack which belongs to the surrounding class template.

§ 4. Impact on the standard

This proposal is a pure deprecation.

No new features are proposed or removed, no semantics are altered, and no existing code is impacted.


```

parameter-declaration-clause:
    parameter-declaration-listopt
    parameter-declaration-list . . .
    parameter-declaration-list , . . .
    . . .

```

Of these productions, the second (highlighted) is deprecated. If the variadic comma was made mandatory, this rule would simply be removed and the rest left intact.

I believe that this editorial change is beneficial to the proposal and improves readability of the *parameter-declaration-clause* production in general.

§ 7. Proposed wording

The proposed wording is relative to [\[N4981\]](#).

In subclause 9.3.4.6 [\[dcl.fct\]](#), modify paragraph 2 as follows:

```

parameter-declaration-clause:
    parameter-declaration-listopt . . .opt
    . . .
    parameter-declaration-listopt
    parameter-declaration-list , . . .
    parameter-declaration-list . . .

```

Update Annex A, subclause A.7 [\[gram.dcl\]](#) accordingly.

In subclause 9.3.4.6 [\[dcl.fct\]](#), modify paragraph 3 as follows:

Where syntactically correct and where ". . ." is not part of an *abstract-declarator*, ~~" , . . . " " . . . "~~ is synonymous with ~~" . . . " " , . . . "~~. A *parameter-declaration-clause* of the form *parameter-declaration-list* . . . is deprecated.

In Annex D [\[depr\]](#), add a subclause:

D.X Non-comma-separated ellipsis parameters [depr.ellipsis.comma]

A *parameter-declaration-clause* of the form *parameter-declaration-list* ... is deprecated.

[Example:

```
void f(int...);           // deprecated  
void g(auto...);         // OK, declares a function parameter pack  
void h(auto.....);      // deprecated
```

— end example]

§ 8. Acknowledgements

I thank Christof Meerwald for contributing proposed wording.

I thank Arthur O’Dwyer for providing extensive editorial feedback, including the suggestion to update *parameter-declaration-clause*.

§ References

§ Normative References

[N4981]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 16 April 2024. URL: <https://wg21.link/n4981>

§ Informative References

[P0281R0]

James Touton. *Remove comma elision in variadic function declarations*. 23 January 2016. URL: <https://wg21.link/p0281r0>

[P1161R3]

Corentin Jabot. *Deprecate uses of the comma operator in subscripting expressions*. 22 February 2019. URL: <https://wg21.link/p1161r3>

[P1219R2]

James Touton. *Homogeneous variadic function parameters*. 6 October 2019. URL: <https://wg21.link/p1219r2>

[P2128R6]

Corentin Jabot, Isabella Muerte, Daisy Hollman, Christian Trott, Mark Hoemmen.

Multidimensional subscript operator. 14 September 2021. URL: <https://wg21.link/p2128r6>